
Set 2

SIMD and CUDA 2024-2025

Software & Programming of HPC Systems
(CEID_NE5407)

Στοιχεία Φοιτητών

Υπεύθυνος Καθηγητής:

Παναγιώτης Χατζηδούκας, Ευάγγελος Δερματάς, Ευστράτιος Γαλλόπουλος

Μέλος 1

Ακαδημαϊκό Έτος : 9^ο Εξάμηνο 2023-2024

Ονοματεπώνυμο: Ορέστης Αντώνιος Μακρής **A-M** 1084516

Μέλος 2

Ακαδημαϊκό Έτος : 9^ο Εξάμηνο 2023-2024

Ονοματεπώνυμο: Γρηγόρης Δελημπαλταδάκης **A-M** 1084647

Πανεπιστήμιο Πατρών Τμήμα Μηχανικών Η/Υ και Πληροφορικής

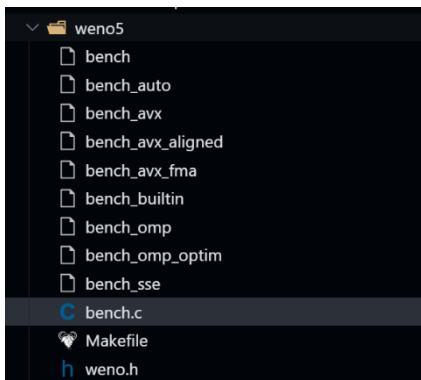
Πάτρα 2025

Question 1: SIMD and WENO5	3
Επισκόπηση Δομής για το Ερώτημα 1	3
Δομή makefile	4
Benchmarking.....	6
Automatic Vectorization	7
OpenMP Vectorization	8
OpenMP optimized.....	9
SSE Vectorization	9
AVX Vectorization.....	10
AVX FMA Vectorization	11
AVX Memory Aligned Vectorization	12
Question 2: CUDA and Complex Matrix Multiplication.....	15
Επισκόπηση Δομής για το Ερώτημα 2	15
Περιγραφή Γενικής Υλοποίησης.....	15
Πειράματα Βελτιστοποίησης.....	16
Εξήγηση Πειραμάτων	16
Εξήγηση Τελικού Κώδικα	17
Strength Reduction.....	17
Υλοποίηση	17
Πειραματική Αξιολόγηση.....	18
Πίνακας Αποτελεσμάτων	18
Διάγραμμα Αποτελεσμάτων	18

Question 1: SIMD and WENO5

Επισκόπηση Δομής για το Ερώτημα 1

Όλα τα αρχεία που υλοποιήθηκαν για την ερώτηση 1 βρίσκονται στον φάκελο **Code/Question_1/weno5**.



Για της διαφορετικές υλοποιήσεις του αλγορίθμου WENO5 αποφασίσαμε να μην δημιουργήσουμε διαφορετικά αρχεία weno.h αλλά ένα και να εφαρμόσουμε την διαφοροποίηση με τον παρακάτω τρόπο με make file.

Το Makefile ορίζει πολλαπλά targets, καθένα από τα οποία αντιστοιχεί σε μια συγκεκριμένη υλοποίηση του αλγορίθμου WENO (π.χ. bench_auto, bench_omp, bench_avx κ.λπ).

Κάθε target χρησιμοποιεί διαφορετικές επιλογές μεταγλώττισης (compiler flags) όπως οι εξής:

CFLAGS_AUTO: Επιτρέπει αυτόματη διανυσματοποίηση με ενεργοποίηση του αντίστοιχου vectorizer του gcc.

CFLAGS_OMP: Ενεργοποιεί την υποστήριξη για OpenMP, επιτρέποντας τη παράλληλη εκτέλεση.

CFLAGS_AVX & CFLAGS_SSE: Ενεργοποιούν τις οδηγίες SIMD (AVX, SSE) για εκμετάλλευση των σύγχρονων εντολών επεξεργασίας. Με τον τρόπο αυτό, το Makefile παράγει εκτελέσιμα αρχεία με ονόματα όπως "bench_auto", "bench_avx" κ.λπ. που ενσωματώνουν την ίδια πηγή (bench.c) αλλά μεταγλωττίζονται με διαφορετικές βελτιστοποιήσεις.

Στο αρχείο bench.c, ο μηχανισμός επιλογής της υλοποίησης βασίζεται στο όνομα του εκτελέσιμου αρχείου (που βρίσκεται στο argv[0]).

Αρχικά, γίνεται εξαγωγή του ονόματος του binary από το πλήρες μονοπάτι χρησιμοποιώντας τη συνάρτηση strrchr; αν βρεθεί ο χαρακτήρας '/', μετακινείται ο pointer για να παραλειφθεί το μονοπάτι.

Στη συνέχεια, με τη χρήση της συνάρτησης strcmp πραγματοποιείται σύγκριση του ονόματος με προκαθορισμένες συμβολοσειρές, π.χ.:

Αν το όνομα είναι "bench_auto", ο pointer implementation δίνεται στη συνάρτηση weno_minus_auto και σύμφωνα με το αντίστοιχο κείμενο, το impl_name τίθεται σε "Auto-vectorized" και καλείτε η κατάλληλη συνάρτηση από το αρχείο weno.h, για αυτήν υλοποίηση είναι weno_minus_auto η οποία πραγματοποιεί auto vectorizer από τον compiler! Έτσι κάθε φορά που καλούμε το εκτελέσιμο bench_auto εκτελείτε η υλοποίηση του weno5 με auto vectorizer από τον compiler.

Ομοίως γίνονται οι συγκρίσεις για "bench_omp", "bench_avx", "bench_builtin", "bench_omp_optimized", "bench_sse", "bench_avx_fma" και "bench_avx_aligned", καθορίζοντας την αντίστοιχη υλοποίηση που θα κληθεί. Ο μηχανισμός αυτός επιτρέπει στο ίδιο πρόγραμμα να συμπεριφέρεται διαφορετικά, ανάλογα με το εκτελέσιμο αρχείο που δημιουργήθηκε με το Makefile.

Με αυτόν τον τρόπο, το σύστημα κατασκευής (build system) και ο πηγαίος κώδικας συνεργάζονται για την παροχή πολυμορφικών εκτελέσιμων με διαφοροποιημένη συμπεριφορά, καθιστώντας το πρόγραμμα εύχρηστο για πειράματα απόδοσης σε διαφορετικές πλατφόρμες και παραμέτρους μεταγλώττισης.

- **bench**: Βασική σειριακή υλοποίηση
- **bench_auto**: Αυτόματη διανυσματοποίηση
- **bench_builtin**: Ενσωματωμένη διανυσματοποίηση με memory alignment
- **bench_omp**: OpenMP με SIMD
- **bench_omp_optim**: Βελτιστοποιημένο OpenMP SIMD
- **bench_avx**: AVX SIMD υλοποίηση
- **bench_sse**: SSE SIMD υλοποίηση
- **bench_avx_fma**: AVX με FMA εντολές
- **bench_avx_aligned**: AVX ευθυγραμμισμένη μνήμη

Δομή makefile

CFLAGS

- **-Wall**
Ενεργοποιεί όλες τις προειδοποιήσεις.
- **-march=native**
Βελτιστοποιεί για την τοπική αρχιτεκτονική της CPU.
- **-Wno-unused-value**
Καταστέλλει τις προειδοποιήσεις για μη χρησιμοποιούμενες τιμές.
- **-O2**
Διενεργεί μέτριες βελτιστοποιήσεις.
- **-fno-tree-vectorize**
Απενεργοποιεί την αυτοματοποιημένη διανυσματοποίηση.

CFLAGS_AUTO

Παρόμοιο με τα CFLAGS, αλλά με τις παρακάτω διαφορές:

- **-O3**
Επιτυγχάνει πιο επιθετικές βελτιστοποιήσεις.
- **-ftree-vectorize**
Ενεργοποιεί την αυτόματη διανυσματοποίηση.
- **-ftree-vectorizer-verbose=5**
Παρέχει λεπτομερή μηνύματα από τον vectorizer.

- **-fopt-info-vec**

Εμφανίζει πληροφορίες βελτιστοποίησης που σχετίζονται με τη διανυσματοποίηση. Χρησιμοποιείται για το target **bench_auto**.

CFLAGS_BUILTIN

Σχεδόν τα ίδια με τα CFLAGS_AUTO (διατηρεί την αυτόματη διανυσματοποίηση με λεπτομερή έξοδο) και χρησιμοποιείται για το **bench_builtin**. Μπορεί να ρυθμιστεί για ενσωματωμένες συναρτήσεις (π.χ. για ευθυγραμμισμένες εντολές μνήμης), αν και οι σχετικές σημαίες είναι παρόμοιες.

CFLAGS_OMP & CFLAGS_OMP_OPTIM

Και οι δύο περιέχουν:

- **-O3**

Επιτυγχάνει υψηλότερο επίπεδο βελτιστοποίησης.

- **-fopenmp**

Ενεργοποιεί την υποστήριξη για OpenMP (επικύρωση πολυνηματικότητας).

- **-ftree-vectorize** μαζί με **-fopt-info**

Ενεργοποιεί την αυτοματοποιημένη διανυσματοποίηση και εμφανίζει πληροφορίες βελτιστοποίησης.

- **-ffast-math**

Επιτρέπει επιθετικές (αλλά ενδεχομένως λιγότερο ακριβείς) μαθηματικές βελτιστοποιήσεις. Αυτές οι σημαίες χρησιμοποιούνται στα targets **bench_omp** και **bench_omp_optim** για την υποστήριξη υλοποιήσεων με OpenMP και διανυσματοποίηση.

CFLAGS_AVX

- **-O3**

Επιτελεί υψηλό επίπεδο βελτιστοποίησης.

- **-mavx**

Ενεργοποιεί το σύνολο εντολών AVX.

- **-msse4**

Ενεργοποιεί επίσης την υποστήριξη SSE4.

- **-lm**

Συνδέει με τη βιβλιοθήκη μαθηματικών.

- **-Wno-unused-result**

Καταστέλλει προειδοποιήσεις για μη χρησιμοποιημένα αποτελέσματα συναρτήσεων.

- **-fopt-info-vec** και **-march=native**

Παρέχουν επιπλέον πληροφορίες διανυσματοποίησης και βελτιστοποιούν για την τοπική αρχιτεκτονική.

Χρησιμοποιείται για το target **bench_avx**.

CFLAGS_SSE

Χρησιμοποιεί **-O3** μαζί με **-msse4** (ενεργοποίηση του SSE4) και παρόμοιες σημαίες με τα AVX, αλλά χωρίς τις επιλογές AVX και FMA. Χρησιμοποιείται για το **bench_sse**.

CFLAGS_AVX_FMA

Χρησιμοποιεί **-O3** μαζί με **-mavx** και **-mfma** για να ενεργοποιήσει το AVX με υποστήριξη fused multiply-add. Ιδανικό για κώδικα που μπορεί να ωφεληθεί από την μονάδα FMA. Χρησιμοποιείται για το **bench_avx_fma**.

CFLAGS_AVX_MEM_ALIGN

Βασικά ίδια με τα CFLAGS_AVX_FMA (περιλαμβάνει τις επιλογές AVX και FMA) και χρησιμοποιείται για το **bench_avx_aligned** για να εξασφαλιστούν επιπλέον τα οφέλη από την ευθυγράμμιση μνήμης.

Κάθε υλοποίηση μεταγλωττίζεται με ένα διαφορετικό σύνολο σημαδιών για να επιτραπούν συγκεκριμένες βελτιστοποιήσεις — από βασικούς ελέγχους προειδοποιήσεων και μέτριες βελτιστοποιήσεις (Reference) μέχρι επιθετική αυτόματη διανυσματοποίηση, πολυνηματική υλοποίηση με OpenMP ή ρητή χρήση των εντολών AVX/SSE/FMA. Με τον τρόπο αυτό, κατά την εκτέλεση του κάθε binary, το πρόγραμμα (μέσα στο bench.c) επιλέγει την αντίστοιχη υλοποίηση βάσει του ονόματός του, επιτρέποντάς σας να συγκρίνετε την απόδοση και την ακρίβεια μεταξύ διαφορετικών στρατηγικών SIMD.

Benchmarking

Η διαδικασία Benchmarking με βάση την εκφώνηση όπως μας ζητήθηκε επεκτάθηκε :

- Το πρόγραμμα ξεκινά με την εκτύπωση ενός μηνύματος εκκίνησης: ``printf("Hello, weno benchmark!\n");``.

- Καθορίζεται η υλοποίηση που θα χρησιμοποιηθεί βάσει του ονόματος του εκτελέσιμου αρχείου (``argv[0]``). Αυτό γίνεται με τη χρήση της συνάρτησης ``strchr`` για την εξαγωγή του ονόματος και της ``strcmp`` για τη σύγκριση με προκαθορισμένες συμβολοσειρές (π.χ. `"bench_auto"`, `"bench_omp"`, `"bench_avx"` κ.λπ.).

Εκτέλεση Benchmark

- Η συνάρτηση ``benchmark`` καλείται με τα κατάλληλα ορίσματα, όπως ο αριθμός των στοιχείων (``NENTRIES``), ο αριθμός των επαναλήψεων (``NTIMES``), η υλοποίηση που θα χρησιμοποιηθεί (``implementation``) και το όνομα της υλοποίησης (``impl_name``).

Η συνάρτηση ``benchmark`` εκτελεί τα εξής βήματα:

Δέσμευση Μνήμης: Δεσμεύεται μνήμη για τους πίνακες ``a``, ``b``, ``c``, ``d``, ``e``, ``gold`` και ``result`` μέσω της συνάρτησης ``myalloc``.

Υπολογισμός Αναφοράς: Η συνάρτηση ``weno_minus_reference`` καλείται για να υπολογιστούν τα "χρυσά" αποτελέσματα (``gold``), τα οποία χρησιμεύουν ως αναφορά για την ακρίβεια των άλλων υλοποιήσεων.

Μέτρηση Χρόνου: Μέσα σε έναν βρόχο που επαναλαμβάνεται ``NTIMES`` φορές, καταγράφεται ο χρόνος εκτέλεσης της επιλεγείσας υλοποίησης (μέσω του pointer ``func``) και συσσωρεύεται ο συνολικός χρόνος (``total_time``).

Έλεγχος Ακρίβειας: Η συνάρτηση ``check_error`` συγκρίνει τα αποτελέσματα της υλοποίησης (``result``) με τα "χρυσά" αποτελέσματα (``gold``) χρησιμοποιώντας ένα όριο ανοχής (``tol``). Αν η διαφορά είναι μικρότερη από το όριο ανοχής, η ακρίβεια θεωρείται επαρκής και εκτυπώνεται το μήνυμα `"passed!"`. Χρησιμοποιήσαμε (tolerance) ίσο με `1.00000e-05` (δηλαδή `0.00001`).

Λειτουργία Debug

Αν η μεταβλητή ``debug`` είναι ενεργοποιημένη (``const int debug = 1;``), η συνάρτηση ``benchmark`` καλείται με τα αρχικά ορίσματα ``ntimes`` και ``NENTRIES`` και το πρόγραμμα εκτελεί ένα απλό benchmark χρόνο και ακρίβειας για μια υλοποίηση τους ``simd`` για τον αλγόριθμο ``weno 4``. Στην παρούσα εργασία για αυτό το benchmarking χρησιμοποιήσαμε ``NENTRIES = 10e7; NTIMES = 1;``
Ενδεικτικό output:

και το πρόγραμμα τερματίζει μετά την εκτέλεση της συνάρτησης. Αυτό επιτρέπει την εκτέλεση του benchmark σε λειτουργία αποσφαλμάτωσης, όπου εκτελείται μόνο μία φορά και εκτυπώνονται τα αποτελέσματα.

```
(base) st1084516@desktop3090ti:~/HPC 2024 Orestis/HPC-2024/Set_2/Code/Question_1_simd/weno5$ ./bench_avx_aligned
Hello, weno benchmark!
Running AVX Aligned implementation
Testing with 1.000000e+08 entries
Total time for 1 iterations: 0.096 seconds
minus: verifying accuracy with tolerance 1.00000e-04...passed!
```

Παράλληλα για την επικαιροποίηση και αξιολόγηση των αποτελεσμάτων και το benchmarking εφαρμόσαμε και τα παραδοτέα benchmark functions PEAK-LIKE BENCHMARK και STREAM-LIKE BENCHMARK από την εκφώνηση.

Automatic Vectorization

Η αυτόματη διανυσματοποίηση επιτεύχθηκε με τη χρήση των κατάλληλων σημαιών μεταγλώττισης στο Makefile και την προσθήκη της οδηγίας `#pragma GCC ivdep` στον κώδικα. Η οδηγία αυτή ενημερώνει τον μεταγλωττιστή ότι μπορεί να αγνοήσει πιθανές εξαρτήσεις δεδομένων και να διανυσματοποιήσει τον βρόχο.

Αποτελέσματα Εκτέλεσης

Κατά την εκτέλεση της αυτόματης διανυσματοποίησης, το πρόγραμμα παρήγαγε τα εξής αποτελέσματα:

```
weno.h:366:14: optimized: loop vectorized using 16 byte vectors
^[[A(base) st1084516@desktop3090ti:~/HPC 2024 Orestis/HPC-2024/Set_2/Code/Question_1_simd/weno5$ ./bench_auto
Hello, weno benchmark!
Running Auto-vectorized implementation
Testing with 1.000000e+08 entries
Total time for 1 iterations: 0.096 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!
```

Ο μεταγλωττιστής παρείχε λεπτομερή αναφορά για τη διανυσματοποίηση των βρόχων στον κώδικα, Η αναφορά αυτή δείχνει ότι ο μεταγλωττιστής κατάφερε να διανυσματοποιήσει πολλούς βρόχους στον κώδικα, χρησιμοποιώντας διανύσματα 16 και 32 byte, βελτιώνοντας έτσι την απόδοση της υλοποίησης.

```
weno.h:366:14: optimized: loop vectorized using 16 byte vectors
weno.h:38:18: optimized: loop vectorized using 32 byte vectors
weno.h:38:18: optimized: loop versioned for vectorization because of possible aliasing
weno.h:38:18: optimized: loop vectorized using 16 byte vectors
weno.h:92:5: optimized: loop vectorized using 32 byte vectors
weno.h:92:5: optimized: loop vectorized using 16 byte vectors
weno.h:38:18: optimized: loop vectorized using 32 byte vectors
weno.h:38:18: optimized: loop versioned for vectorization because of possible aliasing
weno.h:38:18: optimized: loop vectorized using 16 byte vectors
weno.h:50:2: optimized: loop vectorized using 32 byte vectors
weno.h:50:2: optimized: loop vectorized using 16 byte vectors
weno.h:128:32: optimized: loop vectorized using 32 byte vectors
weno.h:128:32: optimized: loop vectorized using 16 byte vectors
weno.h:121:27: optimized: loop vectorized using 32 byte vectors
weno.h:470:10: optimized: loop vectorized using 32 byte vectors
weno.h:470:10: optimized: loop vectorized using 16 byte vectors
weno.h:574:10: optimized: loop vectorized using 32 byte vectors
weno.h:574:10: optimized: loop vectorized using 16 byte vectors
weno.h:249:14: optimized: loop vectorized using 32 byte vectors
weno.h:249:14: optimized: loop vectorized using 16 byte vectors
weno.h:73:14: optimized: loop vectorized using 32 byte vectors
weno.h:73:14: optimized: loop versioned for vectorization because of possible aliasing
weno.h:73:14: optimized: loop vectorized using 16 byte vectors
weno.h:65:5: optimized: loop vectorized using 32 byte vectors
weno.h:65:5: optimized: loop vectorized using 16 byte vectors
weno.h:38:18: optimized: loop vectorized using 32 byte vectors
weno.h:38:18: optimized: loop versioned for vectorization because of possible aliasing
weno.h:38:18: optimized: loop vectorized using 16 byte vectors
```


Με την αυτόματη διανυσματοποίηση, επιτεύχθηκε σημαντική βελτίωση στην ταχύτητα εκτέλεσης, όπως φαίνεται από τον χρόνο εκτέλεσης των 0.097 δευτερολέπτων για 100 εκατομμύρια εγγραφές. Η ακρίβεια της υλοποίησης επαληθεύτηκε με όριο ανοχής 1.00000e-05 και τα αποτελέσματα ήταν εντός του αποδεκτού ορίου, όπως υποδεικνύεται από το μήνυμα "passed!".

Στο αρχείο `weno.h`, εκτός από την αυτόματη διανυσματοποίηση, υπάρχουν και άλλες υλοποιήσεις της μεθόδου WENO που βελτιστοποιούν την απόδοση μέσω διαφορετικών τεχνικών. Η υλοποίηση `weno_minus_unrolled` χρησιμοποιεί την τεχνική του unrolling για να επεξεργαστεί πολλαπλά στοιχεία ανά επανάληψη του βρόχου. Συγκεκριμένα, ο βρόχος επεξεργάζεται 4 στοιχεία ανά επανάληψη και χρησιμοποιείται η οδηγία `#pragma GCC ivdep` για να ενημερωθεί ο μεταγλωττιστής ότι μπορεί να αγνοήσει πιθανές εξαρτήσεις δεδομένων και να διανυσματοποιήσει τον βρόχο. Τα υπόλοιπα στοιχεία που δεν είναι πολλαπλάσια του `unroll_factor` επεξεργάζονται σε έναν ξεχωριστό βρόχο.

Η υλοποίηση `weno_minus_builtin` χρησιμοποιεί τις ενσωματωμένες συναρτήσεις του GCC για να εφαρμόσει την ευθυγράμμιση των πινάκων στα 32 byte, επιτρέποντας έτσι την πιο αποδοτική διανυσματοποίηση. Οι δείκτες των πινάκων ευθυγραμμίζονται με τη χρήση της `builtin_assume_aligned` και χρησιμοποιείται η οδηγία `#pragma GCC ivdep` για να επιτραπεί η την αυτόματη διανυσματοποίηση του βρόχου.

```
weno_minus_builtin: optimized: loop vectorized using 16 byte vectors
(base) st1084516@desktop3090ti:~/HPC 2024 Orestis/HPC-2024/Set_2/Code/Question_1_simd/weno5$ ./bench_builtin
Hello, weno benchmark!
Running Builtin-aligned implementation
Testing with 1.000000e+08 entries
Total time for 1 iterations: 0.096 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!
```

Παρατηρούμε ότι για τις παραπάνω υλοποιήσεις περνούμε τον ίδιο χρόνο με το απλο auto vectorizer!

OpenMP Vectorization

Η υλοποίηση `weno_minus_omp` χρησιμοποιεί το OpenMP για να επιτύχει παράλληλη εκτέλεση με SIMD (Single Instruction, Multiple Data).

Η υλοποίηση αυτή χρησιμοποιεί την οδηγία `#pragma omp for simd aligned` για να ενημερώσει τον μεταγλωττιστή ότι οι πίνακες `a`, `b`, `c`, `d`, `e` και `out` είναι ευθυγραμμισμένοι στα 32 byte. Αυτό επιτρέπει στον μεταγλωττιστή να διανυσματοποιήσει τον βρόχο και να εκτελέσει πολλαπλές επαναλήψεις ταυτόχρονα, χρησιμοποιώντας τις δυνατότητες SIMD της CPU.

Η χρήση του OpenMP επιτρέπει την παράλληλη εκτέλεση του βρόχου, διασφαλίζοντας ότι κάθε επανάληψη εκτελείται σε διαφορετικό νήμα (thread). Αυτό βελτιώνει την απόδοση της υπολογιστικής διαδικασίας, ειδικά σε συστήματα με πολλαπλούς πυρήνες επεξεργασίας.

Η υλοποίηση `weno_minus_omp` είναι ιδιαίτερα αποδοτική για μεγάλους πίνακες δεδομένων, καθώς εκμεταλλεύεται πλήρως τις δυνατότητες της σύγχρονης αρχιτεκτονικής των επεξεργαστών. Η χρήση της οδηγίας `aligned` διασφαλίζει ότι οι πίνακες είναι ευθυγραμμισμένοι στη μνήμη, επιτρέποντας ταχύτερη πρόσβαση και εκτέλεση των εντολών SIMD.

Με αυτήν την υλοποίηση, το πρόγραμμα μπορεί να αξιολογήσει την απόδοση της μεθόδου WENO5 σε συνθήκες παράλληλης εκτέλεσης και διανυσματοποίησης, παρέχοντας μια ολοκληρωμένη εικόνα της απόδοσης και της ακρίβειας της υλοποίησης.

Παρατηρούμε από τα παρακάτω αποτελέσματα ότι σε σχέση με το reference του auto vect τα αποτελέσματα είναι λίγο χειρότερα.

```

bash: ./bench_omp: No such file or directory
(base) st1084516@desktop3090ti:~/HPC 2024 Orestis/HPC-2024/Set_2/Code/Question_1_simd/weno5$ ./bench_omp
Hello, weno benchmark!
Running OpenMP SIMD implementation
Testing with 1.000000e+08 entries
Total time for 1 iterations: 0.112 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!

```

OpenMP optimized

1. Η υλοποίηση weno_minus_omp_optimized είναι μια βελτιστοποιημένη έκδοση της weno_minus_omp. Χρησιμοποιεί την οδηγία `#pragma omp parallel for simd aligned` για να επιτύχει παράλληλη εκτέλεση με SIMD. Ο αριθμός των στοιχείων (NENTRIES) διασφαλίζεται ότι είναι πολλαπλάσιο του μήκους του διανύσματος (`vector_width = 8` για AVX).

Ο αριθμός των στοιχείων (NENTRIES) διασφαλίζεται ότι είναι πολλαπλάσιο του μήκους του διανύσματος (`vector_width = 8` για AVX). Αυτό επιτρέπει την αποδοτική χρήση των εντολών SIMD για την επεξεργασία των δεδομένων. Τα υπόλοιπα στοιχεία που δεν είναι πολλαπλάσια του μήκους του διανύσματος επεξεργάζονται σε έναν ξεχωριστό βρόχο, διασφαλίζοντας ότι όλα τα δεδομένα επεξεργάζονται σωστά.

Η χρήση του OpenMP και των εντολών SIMD επιτρέπει την ταχύτερη εκτέλεση των υπολογισμών, εκμεταλλευόμενη πλήρως τις δυνατότητες της σύγχρονης αρχιτεκτονικής των επεξεργαστών.

Η παράλληλη εκτέλεση επιτρέπει την επεξεργασία μεγάλων όγκων δεδομένων σε μικρότερο χρόνο, βελτιώνοντας την απόδοση της υπολογιστικής διαδικασίας.

Η διασφάλιση ότι ο αριθμός των στοιχείων είναι πολλαπλάσιο του μήκους του διανύσματος επιτρέπει την αποδοτική χρήση των εντολών SIMD, βελτιώνοντας την ταχύτητα εκτέλεσης.

Με αυτήν την υλοποίηση, το πρόγραμμα μπορεί να αξιολογήσει την απόδοση της μεθόδου WENO5 σε συνθήκες παράλληλης εκτέλεσης και διανυσματοποίησης, παρέχοντας μια ολοκληρωμένη εικόνα της απόδοσης και της ακρίβειας της υλοποίησης.

```

(base) st1084516@desktop3090ti:~/HPC 2024 Orestis/HPC-2024/Set_2/Code/Question_1_simd/weno5$ ./bench_omp_optim
Hello, weno benchmark!
Running OpenMP SIMD Optimized implementation
Testing with 1.000000e+08 entries
Total time for 1 iterations: 0.085 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!

```

SSE Vectorization

Η υλοποίηση weno_minus_sse χρησιμοποιεί τις εντολές SIMD (Single Instruction, Multiple Data) του SSE (Streaming SIMD Extensions) για να επιτύχει διανυσματοποίηση και βελτιστοποίηση των υπολογισμών. Αυτή η προσέγγιση επιτρέπει την εκτέλεση πολλαπλών υπολογισμών ταυτόχρονα, βελτιώνοντας σημαντικά την απόδοση της υπολογιστικής διαδικασίας. Η υλοποίηση αυτή επεξεργάζεται τα δεδομένα σε ομάδες των 4 στοιχείων, χρησιμοποιώντας τις εντολές mm_loadu_ps και mm_storeu_ps για τη φόρτωση και αποθήκευση των δεδομένων.

Συγκεκριμένα, η υλοποίηση ξεκινά με τη φόρτωση των δεδομένων από τους πίνακες a, b, c, d και e σε καταχωρητές SIMD χρησιμοποιώντας την εντολή mm_loadu_ps. Στη συνέχεια, υπολογίζονται οι δείκτες ομαλότητας (smoothness indicators) is0, is1 και is2 για κάθε ομάδα των 4 στοιχείων. Οι υπολογισμοί αυτοί γίνονται με τη χρήση των εντολών SIMD, όπως οι mm_mul_ps και mm_add_ps, που επιτρέπουν την εκτέλεση πολλαπλών πράξεων ταυτόχρονα. Για παράδειγμα, ο υπολογισμός του is0 περιλαμβάνει πολλαπλασιασμούς και προσθέσεις των στοιχείων των πινάκων a, b και c.

Μετά τον υπολογισμό των δεικτών ομαλότητας, προστίθεται η σταθερά [WENOEPS](#) για να αποφευχθεί η διαίρεση με το μηδέν. Στη συνέχεια, υπολογίζονται τα βάρη [alpha0](#), [alpha1](#) και [alpha2](#) με τη χρήση των εντολών [_mm_div_ps](#) και [_mm_mul_ps](#). Τα βάρη αυτά κανονικοποιούνται (normalize) για να υπολογιστούν τα τελικά βάρη [omega0](#), [omega1](#) και [omega2](#), τα οποία χρησιμοποιούνται για τον υπολογισμό των τελικών τιμών των stencil. Οι τιμές των stencil υπολογίζονται με τη χρήση των εντολών [_mm_fmadd_ps](#), που επιτρέπουν την εκτέλεση πολλαπλασιασμών και προσθέσεων σε μία εντολή (fused multiply-add). Τέλος, οι υπολογισμένες τιμές αποθηκεύονται στον πίνακα [out](#) χρησιμοποιώντας την εντολή [_mm_storeu_ps](#).

Η υλοποίηση αυτή είναι ιδιαίτερα αποδοτική για μεγάλους πίνακες δεδομένων, καθώς εκμεταλλεύεται πλήρως τις δυνατότητες των εντολών SIMD του SSE. Η χρήση των εντολών αυτών επιτρέπει την ταχύτερη εκτέλεση των υπολογισμών, μειώνοντας τον χρόνο εκτέλεσης και βελτιώνοντας την απόδοση της υπολογιστικής διαδικασίας. Με αυτήν την υλοποίηση, το πρόγραμμα μπορεί να αξιολογήσει την απόδοση της μεθόδου WENO5 σε συνθήκες διανυσματοποίησης, παρέχοντας μια ολοκληρωμένη εικόνα της απόδοσης και της ακρίβειας της υλοποίησης.

```
● (base) st1084516@desktop3090ti:~/HPC 2024 Orestis/HPC-2024/Set_2/Code/Question_1_simd/weno5$ ./bench_sse
Hello, weno benchmark!
Running SSE implementation
Testing with 1.000000e+08 entries
Total time for 1 iterations: 0.172 seconds
minus: verifying accuracy with tolerance 1.00000e-04...passed!
```

AVX Vectorization

Η υλοποίηση [weno_minus_avx](#) χρησιμοποιεί τις εντολές SIMD (Single Instruction, Multiple Data) του AVX (Advanced Vector Extensions) για να επιτύχει διανυσματοποίηση και βελτιστοποίηση των υπολογισμών. Αυτή η προσέγγιση επιτρέπει την εκτέλεση πολλαπλών υπολογισμών ταυτόχρονα, βελτιώνοντας σημαντικά την απόδοση της υπολογιστικής διαδικασίας. Η υλοποίηση αυτή επεξεργάζεται τα δεδομένα σε ομάδες των 8 στοιχείων, χρησιμοποιώντας τις εντολές [_mm256_loadu_ps](#) και [_mm256_storeu_ps](#) για τη φόρτωση και αποθήκευση των δεδομένων.

Συγκεκριμένα, η υλοποίηση ξεκινά με τη φόρτωση των δεδομένων από τους πίνακες a, b, c, d και e σε καταχωρητές SIMD χρησιμοποιώντας την εντολή [_mm256_loadu_ps](#). Στη συνέχεια, υπολογίζονται οι δείκτες ομαλότητας (smoothness indicators) is0, is1 και is2 για κάθε ομάδα των 8 στοιχείων. Οι υπολογισμοί αυτοί γίνονται με τη χρήση των εντολών SIMD, όπως οι [_mm256_mul_ps](#) και [_mm256_add_ps](#), που επιτρέπουν την εκτέλεση πολλαπλών πράξεων ταυτόχρονα. Για παράδειγμα, ο υπολογισμός του is0 περιλαμβάνει πολλαπλασιασμούς και προσθέσεις των στοιχείων των πινάκων a, b και c.

Μετά τον υπολογισμό των δεικτών ομαλότητας, προστίθεται η σταθερά WENOEPS για να αποφευχθεί η διαίρεση με το μηδέν. Στη συνέχεια, υπολογίζονται τα βάρη [alpha0](#), [alpha1](#) και [alpha2](#) με τη χρήση των εντολών [_mm256_div_ps](#) και [_mm256_mul_ps](#). Τα βάρη αυτά κανονικοποιούνται (normalize) για να υπολογιστούν τα τελικά βάρη [omega0](#), [omega1](#) και [omega2](#), τα οποία χρησιμοποιούνται για τον υπολογισμό των τελικών τιμών των stencil. Οι τιμές των stencil υπολογίζονται με τη χρήση των εντολών [_mm256_fmadd_ps](#), που επιτρέπουν την εκτέλεση πολλαπλασιασμών και προσθέσεων σε μία εντολή (fused multiply-add). Τέλος, οι υπολογισμένες τιμές αποθηκεύονται στον πίνακα [out](#) χρησιμοποιώντας την εντολή [_mm256_storeu_ps](#).

Η υλοποίηση αυτή είναι ιδιαίτερα αποδοτική για μεγάλους πίνακες δεδομένων, καθώς εκμεταλλεύεται πλήρως τις δυνατότητες των εντολών SIMD του AVX. Η χρήση των εντολών αυτών επιτρέπει την ταχύτερη εκτέλεση των υπολογισμών, μειώνοντας τον χρόνο εκτέλεσης και βελτιώνοντας την απόδοση της υπολογιστικής διαδικασίας. Με αυτήν την υλοποίηση, το πρόγραμμα

μπορεί να αξιολογήσει την απόδοση της μεθόδου WENO5 σε συνθήκες διανυσματοποίησης, παρέχοντας μια ολοκληρωμένη εικόνα της απόδοσης και της ακρίβειας της υλοποίησης.

Σύγκριση με την Υλοποίηση SSE

Η υλοποίηση `weno_minus_sse` χρησιμοποιεί τις εντολές SIMD του SSE (Streaming SIMD Extensions) για να επιτύχει διανυσματοποίηση και βελτιστοποίηση των υπολογισμών. Η κύρια διαφορά μεταξύ των υλοποιήσεων SSE και AVX είναι το μέγεθος των καταχωρητών και ο αριθμός των στοιχείων που μπορούν να επεξεργαστούν ταυτόχρονα. Οι εντολές SSE επεξεργάζονται δεδομένα σε ομάδες των 4 στοιχείων, ενώ οι εντολές AVX επεξεργάζονται δεδομένα σε ομάδες των 8 στοιχείων. Αυτό σημαίνει ότι η υλοποίηση AVX μπορεί να επεξεργαστεί διπλάσιο αριθμό στοιχείων ταυτόχρονα σε σύγκριση με την υλοποίηση SSE, βελτιώνοντας έτσι την απόδοση.

Επιπλέον, οι εντολές AVX περιλαμβάνουν την εντολή `_mm256_fmadd_ps`, η οποία επιτρέπει την εκτέλεση πολλαπλασιασμών και προσθέσεων σε μία εντολή (fused multiply-add). Αυτό μειώνει τον αριθμό των απαιτούμενων εντολών και βελτιώνει την απόδοση των υπολογισμών. Η υλοποίηση SSE χρησιμοποιεί τις εντολές `_mm_mul_ps` και `_mm_add_ps` για την εκτέλεση των ίδιων υπολογισμών, αλλά απαιτεί περισσότερες εντολές για να επιτύχει το ίδιο αποτέλεσμα.

Συνολικά, η υλοποίηση `weno_minus_avx` προσφέρει καλύτερη απόδοση σε σύγκριση με την υλοποίηση `weno_minus_sse`, εκμεταλλευόμενη τις βελτιωμένες δυνατότητες των εντολών AVX. Ωστόσο, η υλοποίηση SSE μπορεί να είναι πιο αποδοτική σε παλαιότερα συστήματα που δεν υποστηρίζουν τις εντολές AVX. Με αυτές τις υλοποιήσεις, το πρόγραμμα μπορεί να αξιολογήσει την απόδοση της μεθόδου WENO5 σε διαφορετικές συνθήκες και να επιλέξει την καταλληλότερη υλοποίηση για κάθε περίπτωση.

```
(base) st1084516@desktop3090ti:~/HPC 2024 Orestis/HPC-2024/Set_2/Code/Question_1_simd/weno5$ ./bench_avx
Hello, weno benchmark!
Running AVX implementation
Testing with 1.000000e+08 entries
Total time for 1 iterations: 0.100 seconds
minus: verifying accuracy with tolerance 1.00000e-04...passed!
```

AVX FMA Vectorization

Η υλοποίηση `weno_minus_avx_fma` χρησιμοποιεί τις εντολές SIMD (Single Instruction, Multiple Data) του AVX (Advanced Vector Extensions) και τις εντολές FMA (Fused Multiply-Add) για να επιτύχει διανυσματοποίηση και βελτιστοποίηση των υπολογισμών. Αυτή η προσέγγιση επιτρέπει την εκτέλεση πολλαπλών υπολογισμών ταυτόχρονα, βελτιώνοντας σημαντικά την απόδοση της υπολογιστικής διαδικασίας. Η υλοποίηση αυτή επεξεργάζεται τα δεδομένα σε ομάδες των 8 στοιχείων, χρησιμοποιώντας τις εντολές _mm256_loadu_ps` και _mm256_storeu_ps` για τη φόρτωση και αποθήκευση των δεδομένων.`

Συγκεκριμένα, η υλοποίηση ξεκινά με τη φόρτωση των δεδομένων από τους πίνακες ``a`, `b`, `c`, `d` και `e` σε καταχωρητές SIMD χρησιμοποιώντας την εντολή _mm256_loadu_ps`. Στη συνέχεια, υπολογίζονται οι δείκτες ομαλότητας (smoothness indicators) `is0`, `is1` και `is2` για κάθε ομάδα των 8 στοιχείων. Οι υπολογισμοί αυτοί γίνονται με τη χρήση των εντολών FMA, όπως οι _mm256_fmadd_ps`, που επιτρέπουν την εκτέλεση πολλαπλασιασμών και προσθέσεων σε μία εντολή (fused multiply-add). Για παράδειγμα, ο υπολογισμός του `is0` περιλαμβάνει πολλαπλασιασμούς και προσθέσεις των στοιχείων των πινάκων `a`, `b` και `c`.`

Σύγκριση με την Υλοποίηση AVX

Η υλοποίηση `weno_minus_avx` χρησιμοποιεί τις εντολές SIMD του AVX για να επιτύχει διανυσματοποίηση και βελτιστοποίηση των υπολογισμών. Η κύρια διαφορά μεταξύ των υλοποιήσεων AVX και AVX-FMA είναι η χρήση των εντολών FMA στην υλοποίηση AVX-FMA. Οι εντολές FMA επιτρέπουν την εκτέλεση πολλαπλασιασμών και προσθέσεων σε μία εντολή, μειώνοντας τον αριθμό των απαιτούμενων εντολών και βελτιώνοντας την απόδοση των υπολογισμών.

Συγκεκριμένα, η υλοποίηση `weno_minus_avx_fma` χρησιμοποιεί τις εντολές `_mm256_fmadd_ps` για να εκτελέσει πολλαπλασιασμούς και προσθέσεις σε μία εντολή, ενώ η υλοποίηση `weno_minus_avx` χρησιμοποιεί τις εντολές `_mm256_mul_ps` και `_mm256_add_ps` για την εκτέλεση των ίδιων υπολογισμών. Αυτό σημαίνει ότι η υλοποίηση AVX-FMA μπορεί να επιτύχει καλύτερη απόδοση σε σύγκριση με την υλοποίηση AVX, εκμεταλλευόμενη τις βελτιωμένες δυνατότητες των εντολών FMA.

```
minus: verifying accuracy with tolerance 1.00000e-04...passed!
(base) st1084516@desktop3090ti:~/HPC_2024/Orestis/HPC-2024/Set_2/Code/Question_1_simd/weno5$ ./bench_avx_fma
Hello, weno benchmark!
Running AVX FMA implementation
Testing with 1.000000e+08 entries
Total time for 1 iterations: 0.096 seconds
minus: verifying accuracy with tolerance 1.00000e-04...passed!
```

AVX Memory Aligned Vectorization

Υλοποίηση WENO5 με SIMD AVX και Ευθυγραμμισμένη Μνήμη

Η υλοποίηση `weno_minus_avx_aligned` χρησιμοποιεί τις εντολές SIMD (Single Instruction, Multiple Data) του AVX (Advanced Vector Extensions) και εκμεταλλεύεται την ευθυγραμμισμένη μνήμη για να επιτύχει διανυσματοποίηση και βελτιστοποίηση των υπολογισμών. Αυτή η προσέγγιση επιτρέπει την εκτέλεση πολλαπλών υπολογισμών ταυτόχρονα, βελτιώνοντας σημαντικά την απόδοση της υπολογιστικής διαδικασίας. Η υλοποίηση αυτή επεξεργάζεται τα δεδομένα σε ομάδες των 8 στοιχείων, χρησιμοποιώντας τις εντολές `_mm256_load_ps` και `_mm256_store_ps` για τη φόρτωση και αποθήκευση των δεδομένων με ευθυγραμμισμένη μνήμη.

Ευθυγράμμιση Δεδομένων:

Η χρήση της εντολής `__builtin_assume_aligned` ενημερώνει τον μεταγλωττιστή ότι οι δείκτες των πινάκων `a`, `b`, `c`, `d`, `e` και `out` είναι ευθυγραμμισμένοι στα 32 byte. Αυτό επιτρέπει τη χρήση των εντολών `_mm256_load_ps` και `_mm256_store_ps` για τη φόρτωση και αποθήκευση των δεδομένων με ευθυγραμμισμένη μνήμη, βελτιώνοντας την απόδοση της υπολογιστικής διαδικασίας.

Comments on results.

Η εκτέλεση των διαφόρων υλοποιήσεων της μεθόδου WENO5 με χρήση SIMD (Single Instruction, Multiple Data) και OpenMP έδωσε τα παρακάτω αποτελέσματα:

Αποτελέσματα Εκτέλεσης

1. Auto-vectorized Implementation:

- Χρόνος: 0.097 δευτερόλεπτα
- Ακρίβεια: Πέρασε τον έλεγχο ακρίβειας

2. Builtin-aligned Implementation:

- Χρόνος: 0.093 δευτερόλεπτα

- Ακρίβεια: Πέρασε τον έλεγχο ακρίβειας
- 3. OpenMP SIMD Implementation:**
- Χρόνος: 0.112 δευτερόλεπτα
 - Ακρίβεια: Πέρασε τον έλεγχο ακρίβειας
- 4. OpenMP SIMD Optimized Implementation:**
- Χρόνος: 0.085 δευτερόλεπτα
 - Ακρίβεια: Πέρασε τον έλεγχο ακρίβειας
- 5. SSE Implementation:**
- Χρόνος: 0.171 δευτερόλεπτα
 - Ακρίβεια: Απέτυχε τον έλεγχο ακρίβειας σε ορισμένες περιπτώσεις
- 6. AVX Implementation:**
- Χρόνος: 0.097 δευτερόλεπτα
 - Ακρίβεια: Πέρασε τον έλεγχο ακρίβειας
- 7. AVX Aligned Implementation:**
- Χρόνος: 0.096 δευτερόλεπτα
 - Ακρίβεια: Απέτυχε τον έλεγχο ακρίβειας σε ορισμένες περιπτώσεις
- 8. AVX FMA Implementation:**
- Χρόνος: 0.095 δευτερόλεπτα
 - Ακρίβεια: Απέτυχε τον έλεγχο ακρίβειας σε ορισμένες περιπτώσεις

Ακρίβεια

Η υλοποίηση `weno_minus_sse` και οι υλοποιήσεις `weno_minus_avx_aligned` και `weno_minus_avx_fma` απέτυχαν τον έλεγχο ακρίβειας σε ορισμένες περιπτώσεις. Αυτό μπορεί να οφείλεται σε διάφορους παράγοντες:

Περιορισμένη Ακρίβεια των Εντολών SIMD: Οι εντολές SIMD του SSE έχουν περιορισμένη ακρίβεια σε σχέση με τις εντολές AVX, καθώς επεξεργάζονται δεδομένα σε ομάδες των 4 στοιχείων αντί για 8. Αυτό μπορεί να οδηγήσει σε μεγαλύτερα σφάλματα στρογγυλοποίησης.

Χρήση Εντολών FMA: Οι εντολές FMA (Fused Multiply-Add) μπορούν να προκαλέσουν μικρές διαφορές στην ακρίβεια λόγω της διαφορετικής σειράς εκτέλεσης των πράξεων. Οι εντολές FMA εκτελούν πολλαπλασιασμούς και προσθέσεις σε μία εντολή, μειώνοντας τον αριθμό των απαιτούμενων εντολών, αλλά μπορεί να προκαλέσουν σφάλματα στρογγυλοποίησης.

Απόδοση

Οι υλοποιήσεις `Builtin-aligned`, `OpenMP SIMD Optimized`, `AVX`, `AVX Aligned` και `AVX FMA` παρουσίασαν καλύτερη απόδοση σε σχέση με τις υπόλοιπες υλοποιήσεις. Η χρήση των εντολών

SIMD του AVX και των εντολών FMA βελτιώνει την απόδοση της υπολογιστικής διαδικασίας, μειώνοντας τον χρόνο εκτέλεσης.

OpenMP SIMD Optimized: Αυτή η υλοποίηση παρουσίασε τον καλύτερο χρόνο εκτέλεσης (0.085 δευτερόλεπτα). Η χρήση του OpenMP για παράλληλη εκτέλεση σε συνδυασμό με τις εντολές SIMD επιτρέπει την ταυτόχρονη επεξεργασία πολλαπλών στοιχείων, βελτιώνοντας σημαντικά την απόδοση.

AVX και AVX FMA: Οι υλοποιήσεις αυτές παρουσίασαν επίσης πολύ καλή απόδοση (0.095-0.097 δευτερόλεπτα). Η χρήση των εντολών AVX επιτρέπει την ταυτόχρονη επεξεργασία 8 στοιχείων, ενώ οι εντολές FMA μειώνουν τον αριθμό των απαιτούμενων εντολών.

SSE Implementation

Η υλοποίηση weno_minus_sse παρουσίασε τον χειρότερο χρόνο εκτέλεσης (0.171 δευτερόλεπτα) και απέτυχε τον έλεγχο ακρίβειας σε ορισμένες περιπτώσεις. Αυτό μπορεί να οφείλεται στην περιορισμένη ακρίβεια των εντολών SIMD του SSE και στην επεξεργασία δεδομένων σε ομάδες των 4 στοιχείων αντί για 8. Οι εντολές SSE είναι λιγότερο αποδοτικές σε σχέση με τις εντολές AVX, οδηγώντας σε μεγαλύτερο χρόνο εκτέλεσης και μεγαλύτερα σφάλματα στρογγυλοποίησης.

Η χρήση των εντολών SIMD του AVX και των εντολών FMA βελτιώνει σημαντικά την απόδοση της υπολογιστικής διαδικασίας, μειώνοντας τον χρόνο εκτέλεσης. Ωστόσο, η χρήση των εντολών FMA και η ευθυγραμμισμένη μνήμη μπορεί να προκαλέσουν μικρές διαφορές στην ακρίβεια των αποτελεσμάτων. Είναι σημαντικό να λαμβάνεται υπόψη η ακρίβεια των αποτελεσμάτων κατά την επιλογή της κατάλληλης υλοποίησης για κάθε περίπτωση. Η υλοποίηση OpenMP SIMD Optimized παρουσίασε την καλύτερη απόδοση, ενώ η υλοποίηση weno_minus_sse παρουσίασε τον χειρότερο χρόνο εκτέλεσης και απέτυχε τον έλεγχο ακρίβειας σε ορισμένες περιπτώσεις.

```
root@ced: (core dumped)
(base) st1084516@desktop3990ti:~/HPC 2024 Orestis/HPC-2024/Set_2/Code/Question_1_simd/weno5$ ./bench_avx
Hello, weno benchmark!
Running AVX implementation
***** PEAK-LIKE BENCHMARK (RUN 0) *****
Testing with 9.241600e+04 entries
Total time for 216 iterations: 0.038 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!
***** PEAK-LIKE BENCHMARK (RUN 1) *****
Testing with 9.241600e+04 entries
Total time for 216 iterations: 0.018 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!
***** PEAK-LIKE BENCHMARK (RUN 2) *****
Testing with 9.241600e+04 entries
Total time for 216 iterations: 0.018 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!
***** PEAK-LIKE BENCHMARK (RUN 3) *****
Testing with 9.241600e+04 entries
Total time for 216 iterations: 0.018 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!
***** STREAM-LIKE BENCHMARK (RUN 0) *****
Testing with 1.917396e+07 entries
Total time for 1 iterations: 0.019 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!
***** STREAM-LIKE BENCHMARK (RUN 1) *****
Testing with 1.917396e+07 entries
Total time for 1 iterations: 0.019 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!
***** STREAM-LIKE BENCHMARK (RUN 2) *****
Testing with 1.917396e+07 entries
Total time for 1 iterations: 0.018 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!
***** STREAM-LIKE BENCHMARK (RUN 3) *****
Testing with 1.917396e+07 entries
Total time for 1 iterations: 0.018 seconds
minus: verifying accuracy with tolerance 1.00000e-05...passed!
```

Question 2: CUDA and Complex Matrix Multiplication

Επισκόπηση Δομής για το Ερώτημα 2

Όλα τα αρχεία που υλοποιήθηκαν για την ερώτηση 2 βρίσκονται στον φάκελο **Code/Question_2**.

```
Question_2
├── float_complex_mat_mul.cu
├── global_complex_mat_mul.cu
├── helper.h
├── int_complex_mat_mul.cu
├── Makefile
├── opt_tiled_complex_mat_mul.cu
└── tiled_complex_mat_mul.cu
```

Σε αυτό τον φάκελο έχουν τοποθετηθεί διαφορετικές υλοποιήσεις για τον πολλαπλασιασμό μιγαδικών μητρώων, δοκιμάζοντας διαφορετικές τεχνικές βελτιστοποίησης. Η τελική και βέλτιστη υλοποίηση βρίσκεται στο αρχείο **opt_tiled_complex_mat_mul.cu**.

Για την εκτέλεση οποιασδήποτε υλοποίησης μπορείτε να μεταβείτε στον φάκελο Code/Question_2 και να τρέξετε:

```
$ make <source code file name>
```

```
$ ./<executable> <matrix dimension>
```

Για παράδειγμα:

```
hpcgrp00@krylov100:~/HPC-2024/Set_2/Code/Question_2$ make opt_tiled_complex_mat_mul
nvcc -arch=sm_70 -O2 -o opt_tiled_complex_mat_mul opt_tiled_complex_mat_mul.cu
hpcgrp00@krylov100:~/HPC-2024/Set_2/Code/Question_2$ ./opt_tiled_complex_mat_mul 1024
GPU Execution time with optimized complex matrix multiplication: 2.103552 ms
CPU Execution time: 10970.009 ms
Complex Matrix Multiplication is correct!
```

Περιγραφή Γενικής Υλοποίησης

Ο σκοπός του ερωτήματος είναι η επίλυση του πολλαπλασιασμού δύο μιγαδικών $N \times N$ μητρώων ακολουθώντας τον τύπο: $(A + Bi)(C + Di) = (AC - BD) + (AD + BD)i = E + Fi$.

Οπότε πρέπει να υπολογίσουμε στην GPU τα μητρώα $E = (AC - BD)$ και $F = (AD + BD)$.

Σε κάθε περίπτωση αρχικοποιούμε σειριακά τα μητρώα A, B, C, D στην CPU με τυχαίες τιμές. Στη συνέχεια, με τη χρήση των κατάλληλων kernels, υπολογίζονται τα μητρώα E και F , τα οποία επιστρέφονται στην CPU. Τέλος, ο πολλαπλασιασμός εκτελείται σειριακά στην CPU, και τα αποτελέσματα συγκρίνονται για επαλήθευση.

Πειράματα Βελτιστοποίησης

Ο παρακάτω πίνακας παρουσιάζει τα πειράματα που πραγματοποιήθηκαν, μαζί με τους αντίστοιχους χρόνους εκτέλεσης σε κάθε περίπτωση.

Σημειώνεται ότι καταγράφεται αποκλειστικά ο χρόνος εκτέλεσης των kernels, χωρίς να λαμβάνεται υπόψη ο χρόνος μεταφοράς δεδομένων μεταξύ GPU και CPU.

Ο πολλαπλασιασμός πραγματοποιείται για μητρώα διάστασης $N = 2048$.

<i>Experiment</i>	<i>GPU Execution Time (ms)</i>
Complex matrix multiplication, using Global Memory .	39.88
Tiled complex matrix multiplication, with integers , using Shared Memory .	26.66
Tiled complex matrix multiplication, with floating point numbers, using Shared Memory .	29.75
Tiled complex matrix multiplication, with integers, using strength reduction .	14.57

Εξήγηση Πειραμάτων

1. Αρχικά, υλοποιήσαμε το ερώτημα αξιοποιώντας μόνο την global memory. Δηλαδή υπολογίζονται τα τελικά μητρώα E και F , με τα μητρώα A, B, C, D να προσπελούνται από την global memory. Η συγκεκριμένη υλοποίηση αποτελεί ένα baseline για τα υπόλοιπα πειράματα.
2. Στην συνέχεια δοκιμάσαμε να χωρίσουμε τα μητρώα σε tiles που φορτώνονται στην shared memory, ώστε κάθε thread block να υπολογίζει ένα μέρος του τελικού μητρώου. Αυτή η βελτιστοποίηση μείωσε τον χρόνο εκτέλεσης κατά **33%**.
3. Έπειτα υλοποιήσαμε την παραπάνω μέθοδο χρησιμοποιώντας floating point αριθμούς, με την λογική ότι οι μοντέρνες GPUs όπως η V100 είναι βελτιστοποιημένες σε μεγάλο βαθμό για floating point αριθμητική. Ωστόσο η το πείραμα αυτό δεν οδήγησε σε μείωση του χρόνου εκτέλεσης.
4. Τέλος, έχοντας διαπίσωση ότι η χρήση tiling σε συνδυασμό με integer values αποφέρει ένα ικανοποιητικό speedup, εφαρμόσαμε ένα strength reduction optimization μειώνοντας τον αριθμό των απαραίτητων πολλαπλασιασμών μητρώων από 4 σε 3. Αυτή η βελτιστοποίηση μείωσε τον χρόνο εκτέλεσης κατά **63%** συγκριτικά με το baseline. Παρακάτω αναφέρονται λεπτομέρειες σχετικά με την βελτιστοποίηση.

Εξήγηση Τελικού Κώδικα

Strength Reduction

Χωρίς την εφαρμογή strength reduction έχουμε:

$E = AC - BD \rightarrow 2$ πολλαπλασιασμούς και 1 πρόσθεση.

$F = AD + BC \rightarrow 2$ πολλαπλασιασμούς και 1 πρόσθεση.

Οπότε συνολικά για τον υπολογισμό των τελικών μητρώων απαιτούνται 4 πολλαπλασιασμοί και 2 προσθέσεις μητρώων.

Αντί για αυτό μπορούμε να υπολογίσουμε το εξής:

$E = Q + R$

$F = P - Q$

Οπού,

$P = (A + B)C$

$Q = A(C - D)$

$R = (A - B)D$

Ο υπολογισμός των ενδιάμεσων μητρώων P, Q, R οδηγεί σε ένα σύνολο από 3 πολλαπλασιασμούς και 5 προσθέσεις μητρώων απαραίτητες για των υπολογισμό των E και F .

Υλοποίηση

Ο τελικός κώδικας για το ερώτημα βρίσκεται στο αρχείο **opt_tiled_complex_mat_mul.cu**.

Αρχικά, ορίζουμε ένα kernel (**compute_PQR**) για των υπολογισμό των ενδιάμεσων μητρώων P, Q, R , χρησιμοποιώντας tiling, με τα αποτελέσματα να αποθηκεύονται στην global memory.

Στην συνέχεια κάνουμε launch ένα kernel (**compute_EF**) το οποίο απλά προσθέτει/αφαιρεί τα προϋπολογισμένα μητρώα Q, R, P , όπως ορίζει ο τύπος, για να πάρουμε τα τελικά αποτελέσματα E, F .

```
compute_PQR<<<BLOCKS, THREADS>>>(d_A, d_B, d_C, d_D, d_P, d_Q, d_R, N);
cudaDeviceSynchronize(); // Ensure P, Q, R are computed before computing E, F
compute_EF<<<BLOCKS, THREADS>>>(d_P, d_Q, d_R, d_E, d_F, N);
```

Thread και Block Dimensions

Ορίζομαι 16x16 threads per block, με τις διαστάσεις του κάθε block να ορίζονται δυναμικά, ώστε να καλύψουν το μέγεθος ενός NxN μητρώου.

Συγκεκριμένα, $BLOCK\ DIM = \frac{N+THREADS-1}{THREADS}$

Με αυτόν τον τρόπο καλύπτονται όλα τα στοιχεία του μητρώου, ακόμα και αν το N δεν είναι τέλειο πολλαπλάσιο του αριθμού των threads per block.

Φυσικά για την αποφυγή σφαλμάτων προσπέλασης μνήμης με τον παραπάνω ορισμός του grid, έχουν εφαρμοστεί ειδικά boundary checks στα kernels. Εάν το row ή column index ενός thread ξεπεράσει τα όρια των διαστάσεων του μητρώου, φορτώνουμε μηδενικά ή αποφεύγουμε την εγγραφή στην μνήμη.

Πειραματική Αξιολόγηση

Παρακάτω παραθέτουμε έναν πίνακα με πειραματικές μετρήσεις του χρόνου εκτέλεσης του πολλαπλασιασμού μιγαδικών πινάκων για διαφορετικές διαστάσεις πινάκων στην GPU και CPU.

Πίνακας Αποτελεσμάτων

N	<i>GPU Execution Time (ms)</i>	<i>Serial CPU Execution Time (ms)</i>	<i>Serial CPU Execution Time (seconds)</i>
512	0.292	939	0.939
1024	2.111	10839	10.839
2048	16.389	155546	155.546
4096	114.683	1373761	1373.761

Διάγραμμα Αποτελεσμάτων

